In [1]:
```
import pandas as pd
# import pandas as shriniwas
import numpy as np
```

In [11]:
```
data = {
    'Student_id': [1,2,3,4,5,6,7,8,9,10],
    'Name': ['Ayan', 'Priya', 'Sahil', 'Riya', 'Kunal', 'Tanya', 'Rahul', 'Anjali',
    'Age': [18, 20, 21, 22, 25, 18, 18, 19, 23, 24],
    'Gender': ['Female', 'Male', 'Female', 'Female', 'Male', 'Male', 'Female', 'Mal
    'Scores': [[64, 54, 72], [93, 69, 82], [87, 90, 80], [94, 93, 85], [88, 77, 78]
 [58, 96, 61]],
    'Attendance': [92, 95, 85, 88, 96, 80, 97, 78, 93, 89],
    'Grade': ['B', 'C', 'F', 'C', 'F', 'D', 'D', 'C', 'C', 'A']
}
```

In [12]:
```
df = pd.DataFrame(data)
df.head() # Print first 5 rows
```

Out[12]:

	Student_id	Name	Age	Gender	Scores	Attendance	Grade
0	1	Ayan	18	Female	[64, 54, 72]	92	B
1	2	Priya	20	Male	[93, 69, 82]	95	C
2	3	Sahil	21	Female	[87, 90, 80]	85	F
3	4	Riya	22	Female	[94, 93, 85]	88	C
4	5	Kunal	25	Male	[88, 77, 78]	96	F

In [13]:
```
def assign_grade(scores):
    avg_score = np.mean(scores)

    if avg_score > 90:
        return 'A'
    elif avg_score > 80:
        return 'B'
    elif avg_score > 70:
        return 'C'
    elif avg_score > 60:
        return 'D'
    else:
        return 'F'

df['Grade'] = df['Scores'].apply(assign_grade)
```

In [14]:
```
df = pd.DataFrame(data)
df.loc[8, 'Age'] = np.nan
df.loc[29, 'Age'] = np.nan
df.loc[35, 'Age'] = np.nan
df.loc[11, 'Scores'] = None
df.loc[19, 'Scores'] = None
df.loc[9, 'Attendance'] = 105   # invalid percentage
```

```
df.loc[15, 'Grade'] = 'Z'   # invalid grade
df.head(20) # Print first 20 rows
```

Out[14]:

	Student_id	Name	Age	Gender	Scores	Attendance	Grade
0	1.0	Ayan	18.0	Female	[64, 54, 72]	92.0	B
1	2.0	Priya	20.0	Male	[93, 69, 82]	95.0	C
2	3.0	Sahil	21.0	Female	[87, 90, 80]	85.0	F
3	4.0	Riya	22.0	Female	[94, 93, 85]	88.0	C
4	5.0	Kunal	25.0	Male	[88, 77, 78]	96.0	F
5	6.0	Tanya	18.0	Male	[81, 90, 65]	80.0	D
6	7.0	Rahul	18.0	Female	[55, 97, 54]	97.0	D
7	8.0	Anjali	19.0	Male	[54, 68, 97]	78.0	C
8	9.0	Raj	NaN	Male	[92, 67, 76]	93.0	C
9	10.0	Neha	24.0	Female	[58, 96, 61]	105.0	A
29	NaN	NaN	NaN	NaN	NaN	NaN	NaN
35	NaN	NaN	NaN	NaN	NaN	NaN	NaN
11	NaN	NaN	NaN	NaN	None	NaN	NaN
19	NaN	NaN	NaN	NaN	None	NaN	NaN
15	NaN	NaN	NaN	NaN	NaN	NaN	Z

In [15]:

```
missing_values = df.isnull().sum()    #check missing values
invalid_attendance = df[(df['Attendance'] < 0) | (df['Attendance'] > 100)]
invalid_grades = df[~df['Grade'].isin(['A', 'B', 'C', 'D', 'F'])]

print("Missing values:\n", missing_values)
print("Invalid attendance:\n", invalid_attendance)
print("Invalid grades:\n", invalid_grades)
```

```
Missing values:
 Student_id    5
Name           5
Age            6
Gender         5
Scores         5
Attendance     5
Grade          4
dtype: int64
Invalid attendance:
    Student_id  Name   Age  Gender        Scores  Attendance Grade
9         10.0  Neha  24.0  Female  [58, 96, 61]       105.0     A
Invalid grades:
     Student_id Name  Age Gender Scores  Attendance Grade
29          NaN  NaN  NaN    NaN    NaN         NaN   NaN
35          NaN  NaN  NaN    NaN    NaN         NaN   NaN
11          NaN  NaN  NaN    NaN   None         NaN   NaN
19          NaN  NaN  NaN    NaN   None         NaN   NaN
15          NaN  NaN  NaN    NaN    NaN         NaN     Z
```

In [19]:

```python
df['Age'] = df['Age'].fillna(df['Age'].median())

df['Attendance'] = df['Attendance'].apply(
    lambda x: 100 if x > 100 else (0 if x < 0 else x)
)

def handle_invalid_scores(scores):

    # Handle NaN / None
    if scores is None or (isinstance(scores, float) and pd.isna(scores)):
        return [0, 0, 0]

    # Convert numpy array to list
    if isinstance(scores, np.ndarray):
        scores = scores.tolist()

    # If still not a list, replace it
    if not isinstance(scores, list):
        return [0, 0, 0]

    return [max(0, min(100, score)) for score in scores]

df['Scores'] = df['Scores'].apply(handle_invalid_scores)

df['Grade'] = df['Scores'].apply(assign_grade)

df['Grade'] = df['Grade'].apply(
    lambda x: x if x in ['A', 'B', 'C', 'D', 'F'] else 'F'
)

df.head(20)
```

Out[19]:

	Student_id	Name	Age	Gender	Scores	Attendance	Grade
0	1.0	Ayan	18.0	Female	[64, 54, 72]	92.0	D
1	2.0	Priya	20.0	Male	[93, 69, 82]	95.0	B
2	3.0	Sahil	21.0	Female	[87, 90, 80]	85.0	B
3	4.0	Riya	22.0	Female	[94, 93, 85]	88.0	A
4	5.0	Kunal	25.0	Male	[88, 77, 78]	96.0	B
5	6.0	Tanya	65.0	Male	[81, 90, 65]	80.0	C
6	7.0	Rahul	18.0	Female	[55, 97, 54]	97.0	D
7	8.0	Anjali	19.0	Male	[54, 68, 97]	78.0	C
8	9.0	Raj	21.0	Male	[92, 67, 76]	93.0	C
9	10.0	Neha	24.0	Female	[58, 96, 61]	100.0	C
29	NaN	NaN	21.0	NaN	[0, 0, 0]	NaN	F
35	NaN	NaN	21.0	NaN	[0, 0, 0]	NaN	F
11	NaN	NaN	21.0	NaN	[0, 0, 0]	NaN	F
19	NaN	NaN	21.0	NaN	[0, 0, 0]	NaN	F
15	NaN	NaN	21.0	NaN	[0, 0, 0]	NaN	F
10	NaN	NaN	21.0	NaN	[0, 0, 0]	100.0	F
12	NaN	NaN	21.0	NaN	[0, 0, 0]	100.0	F

In [20]:

```
df.loc[5, 'Age'] = 35
df.loc[5, 'Age'] = 50
df.loc[5, 'Age'] = 65
df.loc[10, 'Attendance'] = 200
df.loc[12, 'Attendance'] = 175
df.loc[12, 'Attendance'] = 166

print("DataFrame with Outliers:")
print(df.iloc[5:20])
```

```
DataFrame with Outliers:
    Student_id    Name   Age  Gender        Scores  Attendance Grade
5          6.0   Tanya  65.0    Male  [81, 90, 65]        80.0     C
6          7.0   Rahul  18.0  Female  [55, 97, 54]        97.0     D
7          8.0  Anjali  19.0    Male  [54, 68, 97]        78.0     C
8          9.0     Raj  21.0    Male  [92, 67, 76]        93.0     C
9         10.0    Neha  24.0  Female  [58, 96, 61]       100.0     C
29         NaN     NaN  21.0     NaN     [0, 0, 0]         NaN     F
35         NaN     NaN  21.0     NaN     [0, 0, 0]         NaN     F
11         NaN     NaN  21.0     NaN     [0, 0, 0]         NaN     F
19         NaN     NaN  21.0     NaN     [0, 0, 0]         NaN     F
15         NaN     NaN  21.0     NaN     [0, 0, 0]         NaN     F
10         NaN     NaN  21.0     NaN     [0, 0, 0]       200.0     F
12         NaN     NaN  21.0     NaN     [0, 0, 0]       166.0     F
```

In [21]:
```python
def handle_outliers_iqr(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df[column] = df[column].apply(lambda x: upper_bound if x > upper_bound else (lo

handle_outliers_iqr(df, 'Age')
handle_outliers_iqr(df, 'Attendance')

print(df.iloc[5:20])
```

```
    Student_id    Name   Age  Gender        Scores  Attendance Grade
5          6.0   Tanya  21.0    Male  [81, 90, 65]        80.0     C
6          7.0   Rahul  21.0  Female  [55, 97, 54]        97.0     D
7          8.0  Anjali  21.0    Male  [54, 68, 97]        78.0     C
8          9.0     Raj  21.0    Male  [92, 67, 76]        93.0     C
9         10.0    Neha  21.0  Female  [58, 96, 61]       100.0     C
29         NaN     NaN  21.0     NaN     [0, 0, 0]         NaN     F
35         NaN     NaN  21.0     NaN     [0, 0, 0]         NaN     F
11         NaN     NaN  21.0     NaN     [0, 0, 0]         NaN     F
19         NaN     NaN  21.0     NaN     [0, 0, 0]         NaN     F
15         NaN     NaN  21.0     NaN     [0, 0, 0]         NaN     F
10         NaN     NaN  21.0     NaN     [0, 0, 0]       113.5     F
12         NaN     NaN  21.0     NaN     [0, 0, 0]       113.5     F
```

In [22]:
```python
df['Scaled_Attendance'] = (df['Attendance'] - df['Attendance'].min()) / (df['Attend

print("DataFrame with Min-Max Scaling on 'Attendance':")
print(df[['Attendance', 'Scaled_Attendance']].head(20))
```

```
DataFrame with Min-Max Scaling on 'Attendance':
    Attendance  Scaled_Attendance
0         92.0           0.394366
1         95.0           0.478873
2         85.0           0.197183
3         88.0           0.281690
4         96.0           0.507042
5         80.0           0.056338
6         97.0           0.535211
7         78.0           0.000000
8         93.0           0.422535
9        100.0           0.619718
29         NaN                NaN
35         NaN                NaN
11         NaN                NaN
19         NaN                NaN
15         NaN                NaN
10       113.5           1.000000
12       113.5           1.000000
```

In [ ]: